

PRUEBAS DE COBERTURA

PRUEBAS DEL PAQUETE DOMINIO:

Clase Socio:

Se ha iniciado la fase de pruebas unitarias del paquete dominio, centrando esta primera validación en la clase Socio, al tratarse de una de las entidades principales del sistema de gestión del gimnasio.

El objetivo de estas pruebas ha sido comprobar el correcto funcionamiento de la lógica de negocio asociada a los socios, especialmente en aspectos relacionados con su estado administrativo y la gestión de reservas.

A continuación, se detallan los distintos casos de prueba realizados sobre la clase Socio, especificando para cada uno la entrada utilizada, la salida esperada, la salida real obtenida tras la ejecución y el resultado final de la prueba, con el fin de verificar de forma precisa el correcto funcionamiento de sus métodos principales.

Caso de prueba 1: Verificación de socio moroso (esMoroso())

Entrada:

Socio con estado MOROSO.

Salida esperada:

El método devuelve true.

Salida real:

El método devolvió true.

Resultado:

Correcta

Caso de prueba 2: Verificación de socio activo (esMoroso())

Entrada:

Socio con estado ACTIVO.

Salida esperada:

El método devuelve false.

Salida real:

El método devolvió false.

Resultado:

Correcta

Caso de prueba 3: Actualización de estado (actualizarEstado())

Entrada:

Cambio de estado de ACTIVO a BAJA.

Salida esperada:

El estado del socio se actualiza correctamente a BAJA.

Salida real:

El estado fue actualizado correctamente a BAJA.

Resultado:

Correcta

Caso de prueba 4: Detección de conflicto horario (tieneReservaEnHorario())

Entrada:

Socio con reserva existente de 10:00 a 11:00 y nueva actividad de 10:30 a 11:30.

Salida esperada:

El método devuelve true.

Salida real:

El método devolvió true.

Resultado:

Correcta

Caso de prueba 5: Verificación sin conflicto horario (tieneReservaEnHorario())

Entrada:

Socio con reserva existente de 10:00 a 11:00 y nueva actividad de 12:00 a 13:00.

Salida esperada:

El método devuelve false.

Salida real:

El método devolvió false.

Resultado:

Correcta

Clase Reserva:

Se continuó la fase de pruebas unitarias del paquete dominio mediante la validación de la clase Reserva, elemento fundamental para la gestión de inscripciones de socios en actividades programadas del gimnasio.

El objetivo principal fue comprobar que el sistema administra correctamente los cambios de estado de una reserva, permitiendo tanto su confirmación como su cancelación según las necesidades operativas del gimnasio.

Las pruebas realizadas se centraron en verificar que los métodos encargados de modificar el estado de la reserva funcionan correctamente y reflejan de forma precisa la situación real de cada inscripción.

Los resultados obtenidos confirmaron el correcto funcionamiento de la lógica implementada.

Casos de prueba:

Caso de prueba 1: Confirmación de reserva (confirmarReserva())

Entrada:

Reserva con estado inicial pendiente.

Salida esperada:

El estado cambia a CONFIRMADA.

Salida real:

El estado fue actualizado correctamente a CONFIRMADA.

Resultado:

Correcta

Caso de prueba 2: Cancelación de reserva (cancelarReserva())

Entrada:

Reserva activa.

Salida esperada:

El estado cambia a CANCELADA.

Salida real:

El estado fue actualizado correctamente a CANCELADA.

Resultado:

Correcta

Clase ActividadProgramada:

Se continuó la fase de pruebas unitarias del paquete dominio mediante la validación de la clase ActividadProgramada, responsable de gestionar la planificación concreta de actividades dentro del gimnasio, incluyendo horarios, salas asignadas y control de plazas disponibles.

El objetivo principal de estas pruebas fue verificar el correcto funcionamiento del método hayPlazasDisponibles(), encargado de determinar si una actividad programada dispone aún de capacidad para aceptar nuevas reservas en función del aforo máximo permitido en la sala y el número actual de reservas registradas.

Para ello, se comprobaron distintos escenarios tanto con plazas disponibles como con aforo completo, validando que el sistema responde correctamente en ambos casos.

Los resultados obtenidos fueron satisfactorios, confirmando que la lógica de control de ocupación funciona correctamente y garantiza una adecuada gestión del límite de participantes en las actividades programadas.

Caso de prueba 1: Verificación de plazas disponibles (hayPlazasDisponibles())

Entrada:

Actividad programada con aforo máximo de 2 plazas y 1 reserva registrada.

Salida esperada:

El método devuelve true.

Salida real:

El método devolvió true.

Resultado:

Correcta

Caso de prueba 2: Verificación de aforo completo (hayPlazasDisponibles())

Entrada:

Actividad programada con aforo máximo de 1 plaza y 1 reserva registrada.

Salida esperada:

El método devuelve false.

Salida real:

El método devolvió false.

Resultado:

Correcta

Clase Máquina:

Se continuó la fase de pruebas unitarias del paquete dominio mediante la validación de la clase Máquina, entidad encargada de representar el equipamiento físico del gimnasio y gestionar aspectos fundamentales relacionados con su mantenimiento operativo.

El objetivo principal de estas pruebas fue verificar el correcto funcionamiento del método `necesitaMantenimiento()`, cuya finalidad es determinar si una máquina requiere revisión técnica en función de su antigüedad y del tiempo transcurrido desde su último mantenimiento registrado.

Para ello, se analizaron distintos escenarios contemplando tanto máquinas con mantenimiento al día como máquinas que superan los periodos máximos permitidos de revisión.

Los resultados obtenidos fueron satisfactorios, confirmando que el sistema aplica correctamente las reglas de mantenimiento preventivo y permite detectar adecuadamente cuándo una máquina requiere intervención técnica.

Caso de prueba 1: Máquina sin necesidad de mantenimiento (`necesitaMantenimiento()`)

Entrada:

Máquina con menos de 3 años de antigüedad y último mantenimiento realizado hace menos de 12 meses.

Salida esperada:

El método devuelve `false`.

Salida real:

El método devolvió `false`.

Resultado:

Correcta

Caso de prueba 2: Máquina que requiere mantenimiento (`necesitaMantenimiento()`)

Entrada:

Máquina con más de 3 años de antigüedad y último mantenimiento realizado hace más de 6 meses.

Salida esperada:

El método devuelve `true`.

Salida real:

El método devolvió true.

Resultado:

Correcta

PRUEBAS DEL PAQUETE BD_DAO:

Se continúa la fase de pruebas unitarias del paquete bd_dao mediante la validación de la clase SocioDAO, encargada de gestionar la persistencia de los datos de socios dentro de la base de datos del sistema.

El objetivo principal de estas pruebas es comprobar que las operaciones fundamentales de acceso y manipulación de datos (CRUD) funcionan correctamente, garantizando que el sistema permite insertar, consultar, verificar existencia, actualizar y eliminar registros de socios de forma segura, precisa y coherente.

Estas comprobaciones resultan esenciales para asegurar la integridad de la información almacenada y validar el correcto funcionamiento de uno de los módulos más importantes del sistema de gestión del gimnasio.

Los resultados obtenidos confirman que la clase SocioDAO gestiona adecuadamente las operaciones de persistencia relacionadas con los socios.

Clase SocioDAO:**Caso de prueba 1: Inserción de socio (insertarSocio())****Entrada:**

Se inserta un nuevo socio con DNI 99999999Z y datos personales básicos.

Salida esperada:

El socio queda registrado correctamente en la base de datos.

Salida real:

El socio se inserta correctamente.

Resultado:

Correcta

Caso de prueba 2: Búsqueda de socio (buscarPorDni())**Entrada:**

Se realiza una búsqueda mediante el DNI 99999999Z.

Salida esperada:

El sistema recupera correctamente los datos del socio.

Salida real:

El socio se localiza correctamente en la base de datos.

Resultado:

Correcta

Caso de prueba 3: Comprobación de existencia (existeSocio())**Entrada:**

Se verifica la existencia del socio con DNI 99999999Z.

Salida esperada:

El método devuelve true.

Salida real:

El método devuelve true.

Resultado:

Correcta

Caso de prueba 4: Actualización de socio (actualizarSocio())**Entrada:**

Se modifica el nombre del socio de Juan a Carlos.

Salida esperada:

Los datos del socio se actualizan correctamente.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Caso de prueba 5: Eliminación de socio (eliminarSocio())**Entrada:**

Se elimina el socio con DNI 99999999Z.

Salida esperada:

El socio deja de existir en la base de datos.

Salida real:

El socio se elimina correctamente.

Resultado:

Correcta

Clase SalaDAO:

Se continúa la fase de pruebas unitarias del paquete bd_dao mediante la validación de la clase SalaDAO, encargada de gestionar la persistencia de la información relacionada con las salas del gimnasio dentro de la base de datos del sistema.

Los resultados obtenidos confirman que la clase SalaDAO gestiona correctamente las operaciones de persistencia asociadas a la administración de salas

Caso de prueba 1: Inserción de sala (insertarSala())**Entrada:**

Se inserta una nueva sala con nombre identificativo, dimensiones y aforo máximo.

Salida esperada:

La sala queda registrada correctamente en la base de datos.

Salida real:

La sala se inserta correctamente.

Resultado:

Correcta

Caso de prueba 2: Búsqueda de sala (buscarPorNombre())**Entrada:**

Se realiza una búsqueda utilizando el nombre de la sala registrada.

Salida esperada:

El sistema recupera correctamente los datos de la sala.

Salida real:

La sala se localiza correctamente en la base de datos.

Resultado:

Correcta

Caso de prueba 3: Comprobación de existencia (existeSala())

Entrada:

Se verifica la existencia de una sala previamente registrada.

Salida esperada:

El método devuelve true.

Salida real:

El método devuelve true.

Resultado:

Correcta

Caso de prueba 4: Actualización de sala (actualizarSala())

Entrada:

Se modifica el aforo máximo de la sala registrada.

Salida esperada:

Los datos de la sala se actualizan correctamente en la base de datos.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Caso de prueba 5: Eliminación de sala (eliminarSala())

Entrada:

Se elimina la sala previamente registrada.

Salida esperada:

La sala deja de existir en la base de datos.

Salida real:

La sala se elimina correctamente.

Resultado:

Correcta

Clase ActividadDAO:

Se continúa la fase de pruebas unitarias del paquete bd_dao mediante la validación de la clase ActividadDAO, encargada de gestionar la persistencia de la información relacionada con las actividades ofertadas por el gimnasio dentro de la base de datos del sistema.

Los resultados obtenidos confirman que la clase ActividadDAO gestiona correctamente las operaciones de persistencia asociadas a la administración de actividades.

Caso de prueba 1: Inserción de actividad (insertarActividad())

Entrada:

Se inserta una nueva actividad con nombre identificativo, descripción y nivel.

Salida esperada:

La actividad queda registrada correctamente en la base de datos.

Salida real:

La actividad se inserta correctamente.

Resultado:

Correcta

Caso de prueba 2: Búsqueda de actividad (buscarPorNombre())

Entrada:

Se realiza una búsqueda utilizando el nombre de la actividad registrada.

Salida esperada:

El sistema recupera correctamente los datos de la actividad.

Salida real:

La actividad se localiza correctamente en la base de datos.

Resultado:

Correcta

Caso de prueba 3: Comprobación de existencia (existeActividad())

Entrada:

Se verifica la existencia de una actividad previamente registrada.

Salida esperada:

El método devuelve true.

Salida real:

El método devuelve true.

Resultado:

Correcta

Caso de prueba 4: Actualización de actividad (actualizarActividad())**Entrada:**

Se modifica la descripción de una actividad registrada.

Salida esperada:

Los datos de la actividad se actualizan correctamente en la base de datos.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Caso de prueba 5: Eliminación de actividad (eliminarActividad())**Entrada:**

Se elimina la actividad previamente registrada.

Salida esperada:

La actividad deja de existir en la base de datos.

Salida real:

La actividad se elimina correctamente.

Resultado:

Correcta

Clase ReservaDAO:

Se continúa la fase de pruebas unitarias del paquete bd_dao mediante la validación de la clase ReservaDAO, responsable de gestionar la persistencia de las reservas o inscripciones de socios en actividades programadas dentro de la base de datos del sistema.

Los resultados obtenidos confirman que la clase ReservaDAO permite gestionar correctamente las operaciones de persistencia relacionadas con las inscripciones.

Caso de prueba 1: Inserción de reserva (insertarReserva())

Entrada:

Se registra una nueva reserva asociando un socio existente con una actividad programada válida.

Salida esperada:

La reserva queda almacenada correctamente en la base de datos.

Salida real:

La reserva se inserta correctamente.

Resultado:

Correcta

Caso de prueba 2: Búsqueda de reserva (buscarPorId())

Entrada:

Se realiza una búsqueda mediante el identificador de una reserva existente.

Salida esperada:

El sistema recupera correctamente los datos de la reserva.

Salida real:

La reserva se localiza correctamente en la base de datos.

Resultado:

Correcta

Caso de prueba 3: Comprobación de existencia (existeReserva())

Entrada:

Se verifica la existencia de una reserva previamente registrada.

Salida esperada:

El método devuelve true.

Salida real:

El método devuelve true.

Resultado:

Correcta

Caso de prueba 4: Eliminación de reserva (eliminarReserva())

Entrada:

Se elimina una reserva previamente registrada.

Salida esperada:

La reserva deja de existir en la base de datos.

Salida real:

La reserva se elimina correctamente.

Resultado:

Correcta

Clase ActividadProgramadaDAO:

Se continúa la fase de pruebas unitarias del paquete bd_dao mediante la validación de la clase ActividadProgramadaDAO, encargada de gestionar la persistencia de las actividades programadas dentro de la base de datos del sistema.

El objetivo principal de estas pruebas es comprobar el correcto funcionamiento de las operaciones fundamentales de inserción, consulta, verificación de existencia y actualización de actividades programadas, garantizando que el sistema administra correctamente la planificación temporal de actividades, así como su relación con actividades base, salas y entrenadores.

Los resultados obtenidos confirman que la clase ActividadProgramadaDAO permite gestionar correctamente las operaciones de persistencia relacionadas con la programación de actividades.

Caso de prueba 1: Inserción de actividad programada (insertarActividadProgramada())

Entrada:

Se registra una nueva actividad programada asociando una actividad existente, una sala válida, un entrenador y un intervalo horario definido.

Salida esperada:

La actividad programada queda almacenada correctamente en la base de datos.

Salida real:

La actividad programada se inserta correctamente.

Resultado:

Correcta

Caso de prueba 2: Búsqueda de actividad programada (buscarPorId())**Entrada:**

Se realiza una búsqueda mediante el identificador de una actividad programada existente.

Salida esperada:

El sistema recupera correctamente los datos de la actividad programada.

Salida real:

La actividad programada se localiza correctamente en la base de datos.

Resultado:

Correcta

Caso de prueba 3: Comprobación de existencia (existeActividadProgramada())**Entrada:**

Se verifica la existencia de una actividad programada previamente registrada.

Salida esperada:

El método devuelve true.

Salida real:

El método devuelve true.

Resultado:

Correcta

Caso de prueba 4: Actualización de actividad programada (actualizarActividadProgramada())**Entrada:**

Se modifica la fecha de finalización de una actividad programada existente.

Salida esperada:

Los datos de la actividad programada se actualizan correctamente en la base de datos.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Clase MaquinaDAO:

Se continúa la fase de pruebas unitarias del paquete bd_dao mediante la validación de la clase MaquinaDAO, encargada de gestionar la persistencia de la información relacionada con la maquinaria del gimnasio dentro de la base de datos del sistema.

Los resultados obtenidos confirman que la clase MaquinaDAO permite gestionar correctamente las operaciones de persistencia relacionadas con las máquinas.

Caso de prueba 1: Inserción de máquina (insertarMaquina())

Entrada:

Se registra una nueva máquina asociando tipo, marca, número de serie, fechas de compra y mantenimiento, estado operativo y sala asignada.

Salida esperada:

La máquina queda almacenada correctamente en la base de datos.

Salida real:

La máquina se inserta correctamente.

Resultado:

Correcta

Caso de prueba 2: Obtención de máquinas por sala (obtenerMaquinasPorSala())

Entrada:

Se realiza una consulta de máquinas asociadas a una sala existente.

Salida esperada:

El sistema recupera correctamente la lista de máquinas registradas en dicha sala.

Salida real:

Las máquinas se recuperan correctamente.

Resultado:

Correcta

Caso de prueba 3: Actualización de estado de máquina (actualizarEstado())

Entrada:

Se modifica el estado operativo de una máquina registrada.

Salida esperada:

El estado de la máquina se actualiza correctamente en la base de datos.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Clase EntrenadorDAO:

Se finaliza la fase principal de pruebas unitarias del paquete bd_dao mediante la validación de la clase EntrenadorDAO, encargada de gestionar la persistencia de los datos relacionados con los entrenadores del gimnasio dentro de la base de datos del sistema.

Los resultados obtenidos confirman que la clase EntrenadorDAO gestiona correctamente las operaciones de persistencia asociadas a la administración de entrenadores.

Caso de prueba 1: Inserción de entrenador (insertarEntrenador())**Entrada:**

Se inserta un nuevo entrenador con DNI y datos personales básicos.

Salida esperada:

El entrenador queda registrado correctamente en la base de datos.

Salida real:

El entrenador se inserta correctamente.

Resultado:

Correcta

Caso de prueba 2: Búsqueda de entrenador (buscarPorDni())**Entrada:**

Se realiza una búsqueda mediante el DNI del entrenador registrado.

Salida esperada:

El sistema recupera correctamente los datos del entrenador.

Salida real:

El entrenador se localiza correctamente en la base de datos.

Resultado:

Correcta

Caso de prueba 3: Comprobación de existencia (existeEntrenador())

Entrada:

Se verifica la existencia del entrenador previamente registrado.

Salida esperada:

El método devuelve true.

Salida real:

El método devuelve true.

Resultado:

Correcta

Caso de prueba 4: Actualización de entrenador (actualizarEntrenador())

Entrada:

Se modifica el nombre del entrenador registrado.

Salida esperada:

Los datos del entrenador se actualizan correctamente en la base de datos.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Caso de prueba 5: Eliminación de entrenador (eliminarEntrenador())

Entrada:

Se elimina el entrenador previamente registrado.

Salida esperada:

El entrenador deja de existir en la base de datos.

Salida real:

El entrenador se elimina correctamente.

Resultado:

Correcta

PRUEBAS DEL PAQUETE GESTIÓN:

Una vez completadas las pruebas unitarias correspondientes a las clases de dominio y persistencia (DAO), se inicia la fase de validación del paquete gestion, encargado de implementar la lógica de negocio principal del sistema.

El objetivo de esta etapa es comprobar que las reglas funcionales del gimnasio se ejecutan correctamente, verificando que las distintas operaciones de alta, búsqueda, actualización, eliminación y validación empresarial funcionan de forma coherente antes de interactuar con la base de datos.

Estas pruebas resultan especialmente importantes, ya que la capa de gestión actúa como núcleo coordinador entre la interfaz de usuario y la persistencia, asegurando que todas las acciones del sistema respetan las restricciones funcionales definidas en el proyecto.

La validación del paquete gestion permite garantizar que el comportamiento global del sistema responde correctamente a situaciones reales de uso.

Clase UsuarioGestion:

Se realizan pruebas unitarias sobre la clase UsuarioGestion para verificar el correcto funcionamiento de la lógica de negocio relacionada con la gestión de usuarios del sistema, especialmente en operaciones de alta, búsqueda, actualización y eliminación de socios.

Caso de prueba 1: Alta de socio (altaSocio())

Entrada:

Se registra un nuevo socio con datos válidos.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

El alta se realiza correctamente.

Resultado:

Correcta

Caso de prueba 2: Alta de socio duplicado (altaSocio())

Entrada:

Se intenta registrar un socio ya existente.

Salida esperada:

El sistema devuelve ResultadoGestion.YA_EXISTE.

Salida real:

El sistema detecta correctamente el duplicado.

Resultado:

Correcta

Caso de prueba 3: Búsqueda de usuario (buscarUsuarioPorDni())**Entrada:**

Se busca un usuario existente mediante DNI.

Salida esperada:

El sistema localiza correctamente al usuario.

Salida real:

La búsqueda se realiza correctamente.

Resultado:

Correcta

Caso de prueba 4: Actualización de socio (actualizarSocio())**Entrada:**

Se modifican datos de un socio existente.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Caso de prueba 5: Eliminación de socio (eliminarSocio())**Entrada:**

Se elimina un socio existente.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

La eliminación se realiza correctamente.

Resultado:

Correcta

Clase GestionActividades:

Se realizan pruebas unitarias sobre la clase GestionActividades para verificar el correcto funcionamiento de la lógica de negocio relacionada con la gestión de actividades del sistema, especialmente en operaciones de alta, validación de duplicados, búsqueda, actualización y eliminación.

Caso de prueba 1: Alta de actividad (altaActividad())**Entrada:**

Se registra una nueva actividad con datos válidos.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

El alta se realiza correctamente.

Resultado:

Correcta

Caso de prueba 2: Alta de actividad duplicada (altaActividad())**Entrada:**

Se intenta registrar una actividad ya existente.

Salida esperada:

El sistema devuelve ResultadoGestion.YA_EXISTE.

Salida real:

El sistema detecta correctamente el duplicado.

Resultado:

Correcta

Caso de prueba 3: Búsqueda de actividad (buscarActividadPorNombre())

Entrada:

Se busca una actividad existente por nombre.

Salida esperada:

El sistema localiza correctamente la actividad.

Salida real:

La búsqueda se realiza correctamente.

Resultado:

Correcta

Caso de prueba 4: Actualización de actividad (actualizarActividad())

Entrada:

Se modifican los datos de una actividad existente.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Caso de prueba 5: Eliminación de actividad (eliminarActividad())

Entrada:

Se elimina una actividad existente.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

La eliminación se realiza correctamente.

Resultado:

Correcta

Clase GestionReservaTest:

Se realizan pruebas unitarias sobre la clase GestionReservas para verificar el correcto funcionamiento de la lógica de negocio relacionada con la consulta y cancelación de reservas dentro del sistema.

Caso de prueba 1: Búsqueda de reserva (buscarReservaPorId())

Entrada:

Se busca una reserva mediante su identificador.

Salida esperada:

El sistema localiza correctamente la reserva si existe.

Salida real:

La búsqueda se realiza correctamente según disponibilidad del registro.

Resultado:

Correcta

Caso de prueba 2: Cancelación de reserva (cancelarReserva())

Entrada:

Se solicita la cancelación de una reserva existente.

Salida esperada:

El sistema devuelve ResultadoGestion.OK o controla correctamente si la reserva ya no existe.

Salida real:

La cancelación se gestiona correctamente.

Resultado:

Correcta

Clase GestionSalaMaquina:

Se realizan pruebas unitarias sobre la clase GestionSalaMaquina para verificar el correcto funcionamiento de la lógica de negocio relacionada con la gestión de salas del gimnasio, incluyendo altas, validación de duplicados, búsquedas, actualizaciones y eliminaciones.

Caso de prueba 1: Alta de sala (altaSala())

Entrada:

Se registra una nueva sala con datos válidos.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

El alta se realiza correctamente.

Resultado:

Correcta

Caso de prueba 2: Alta de sala duplicada (altaSala())

Entrada:

Se intenta registrar una sala ya existente.

Salida esperada:

El sistema devuelve ResultadoGestion.YA_EXISTE.

Salida real:

El sistema detecta correctamente el duplicado.

Resultado:

Correcta

Caso de prueba 3: Búsqueda de sala (buscarSalaPorNombre())

Entrada:

Se busca una sala existente mediante su nombre.

Salida esperada:

El sistema localiza correctamente la sala.

Salida real:

La búsqueda se realiza correctamente.

Resultado:

Correcta

Caso de prueba 4: Actualización de sala (actualizarSala())

Entrada:

Se modifican los datos de una sala existente.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

La actualización se realiza correctamente.

Resultado:

Correcta

Caso de prueba 5: Eliminación de sala (eliminarSala())

Entrada:

Se elimina una sala existente.

Salida esperada:

El sistema devuelve ResultadoGestion.OK.

Salida real:

La eliminación se realiza correctamente.

Resultado:

Correcta